TECHNICAL DOCUMENTATION

qp: A Systematic Trading Research Pipeline

Architecture, Design Rationale, and First Results

Batbandi Gerelkhuu

Compiled 4 September 2026 · v0.1.0

This document describes a research and paper-trading system, not a live trading strategy. Every headline number in Chapter 8 is measured on synthetic data with an injected, exaggerated signal so the machinery is visible end to end — it is not evidence that the included strategies make money on real markets. See Chapter 10 for the caveats this document insists you read.

Repository: `quant-pipeline/` (sibling to the Mongolia blueprint and the PyPSA thesis work). Code: `src/qp/`. Tests: `tests/` (13 passing). Agent layer: `mcp/server.py`.

Contents

Executive Summary **1**

1 System Architecture **3**
1.1 The stage pipeline . . . 3
1.2 Orchestration . . . 3
1.3 Why this shape . . . 4

2 The Data Layer **5**
2.1 Contract . . . 5
2.2 Four sources, one always working . . . 5
2.3 The synthetic generator . . . 5

3 Feature Engineering **7**
3.1 Causal, token-resolved indicators . . . 7
3.2 Building an ML dataset without leaking the future . . . 7

4 Signal Models **9**
4.1 The contract . . . 9
4.2 Rule-based signals . . . 9
4.3 A walk-forward ML signal . . . 9
4.4 An optional sequence model . . . 9

5 Validation Methodology **11**
5.1 Walk-forward . . . 11
5.2 Purged K-fold . . . 11

6 The Backtest Engine **13**
6.1 Timeline convention . . . 13
6.2 Position sizing . . . 13
6.3 Cost model . . . 14
6.4 Risk limits and the drawdown kill-switch . . . 14
6.5 Metrics . . . 14

7 Execution and the Live Loop **16**
7.1 The broker contract . . . 16
7.2 The paper broker . . . 16
7.3 The live loop . . . 16
7.4 The real-broker stub . . . 16

8 Results on the Bundled Demo **18**
8.1 The demo run . . . 18
8.2 Rule versus ML, same data, same costs . . . 18

8.3 Why the rule beat the model here 19

9 The Agent Layer: MCP, and What's Next **20**
9.1 Why an MCP server on top of a CLI 20
9.2 What is exposed today 20
9.3 Roadmap 20

10 Honest Limitations **22**
10.1 Everything in Chapter 8 is synthetic 22
10.2 Retail systematic trading is a near-zero-sum game 22
10.3 Why the LSTM (and the GBR) should not be trusted here 22
10.4 What this project is actually worth 23

A Configuration Reference and File Map **24**
A.1 `RunConfig` schema 24
A.2 File map 25
A.3 Reproducing this document's numbers 25

Executive Summary

qp is a full systematic-trading research pipeline: configuration, data ingestion, feature engineering, alpha signals, walk-forward validation, a cost-aware vectorised backtester, pre-trade risk limits, a paper-trading broker, reporting, and a live-loop orchestrator – with a Model Context Protocol (MCP) server on top so an agent can drive the whole thing. It is roughly 2,400 lines of Python across 30 modules, has 13 passing tests, and runs clean under `ruff`.

Why this exists

Not to make money trading. Retail systematic trading is a near-zero-sum game against firms with better data, lower fees, and colocation; Chapter 10 makes that case in full. qp was built for two reasons instead:

1. It is the portfolio artefact for an energy/commodity trading-desk application – a clean backtester and a market model map directly onto what those roles actually do (Chapter 8 of the companion Mongolia blueprint; Part I of this document shows the mechanics).
2. It teaches market microstructure that compounds: no-look-ahead discipline, realistic transaction costs, walk-forward validation, and why an LSTM usually loses to a simple rule once costs are charged.

What is in it

Stage	What it does
`config`	one YAML fully describes a run; typed `pydantic` models per stage
`data`	`DataSource` contract: synthetic (deterministic default), CSV, yfinance, stooq; parquet cache
`features`	causal technical indicators, token-resolved (`mom_60`, `rsi_14`); leak-free ML dataset builder
`signals`	momentum, mean-reversion, cross-sectional momentum, a walk-forward ML model, an optional LSTM
`validation`	walk-forward and purged K-fold splits with an embargo
`backtest`	vectorised, one-bar execution lag, $\sqrt{w^\top \Sigma w}$ vol targeting, full cost model
`risk`	per-name and gross exposure caps, a drawdown kill-switch with cooldown
`execution`	a JSON-persisted paper broker (default); a real-broker adapter left as a deliberate stub
`report`	a matplotlib tearsheet plus machine-readable metrics
`live`	the pull → features → signal → target → reconcile → orders loop
`mcp`	the pipeline exposed as seven agent-callable tools

Headline result

On the bundled synthetic demo, a simple cross-sectional momentum rule delivers a Sharpe ratio of 0.62 after realistic costs; a gradient-boosted ML model on the same data, engineered to look attractive, delivers roughly −0.1. That gap is the point of this whole exercise, not an

embarrassment: Chapter 8 shows the numbers and Chapter 10 explains why it is the expected and correct outcome.

CHAPTER 1

System Architecture

1.1 The stage pipeline

A run is one YAML file (`configs/*.yaml`) interpreted by a chain of pure, testable stages. Each stage reads only the sub-model of `RunConfig` it needs, so configs stay flat and reviewable, and each stage can be tested in isolation with synthetic inputs.

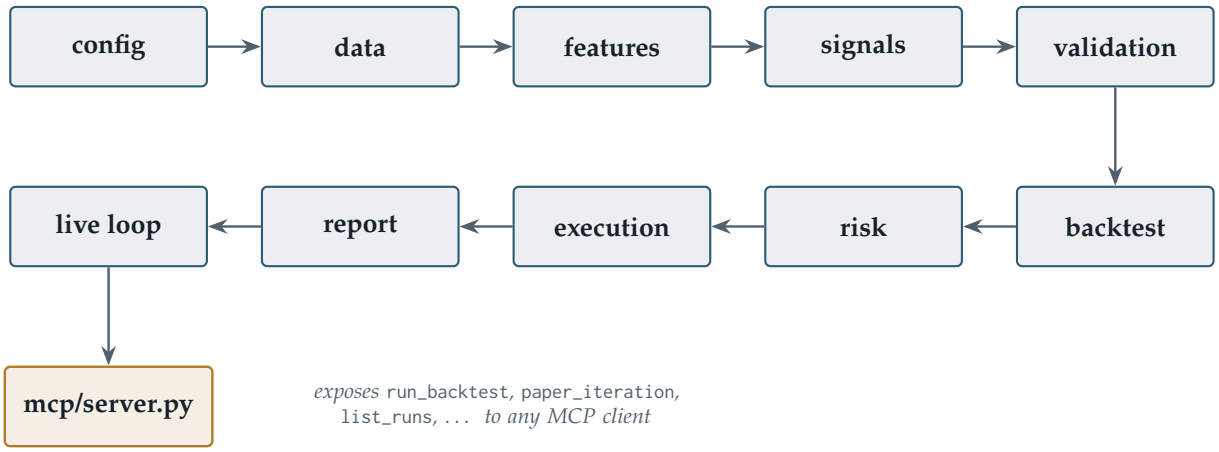


Figure 1.1: The eleven modules of `src/qp/`, in call order. The bottom row reads right to left to keep the diagram compact; data still flows backtest → risk → execution → report → live.

1.2 Orchestration

`qp.pipeline.run_pipeline(cfg)` is the single entry point used by the CLI's `demo` and `backtest` commands:

```
long_df = load_prices(cfg.data) # data
close = to_wide(long_df, "close")
signal = make_signal(cfg.signal, cfg.features, cfg.validation)
scores = signal.generate(close) # features + signals (fused)
result = run_backtest(close, scores, cfg) # validation + backtest + risk
write_tearsheet(result, cfg.run_dir()) # report
```

`qp.live.loop.run_once(cfg)` reuses the same `data`, `features`, and `signals` code, then diverges into `execution` instead of `backtest` – the live loop and the backtester are guaranteed to compute the same target weights from the same signal, because they call the same functions.

1.3 Why this shape

Design rationale

Every stage is a pure function or a small class with one public method (`generate`, `fetch`, `rebalance_to`). Nothing reaches into another stage's internals. That buys three things directly relevant to career capital: (1) each stage is unit-testable against synthetic data without a network or a broker; (2) a new signal, data source, or broker is additive, not a rewrite; (3) the same code path serves the backtester and the live loop, so there is no "it worked in backtest but not live" class of bug from divergent implementations.

CHAPTER 2

The Data Layer

2.1 Contract

Every source implements `DataSource.fetch(symbols, start, end) -> DataFrame` and returns a long OHLCV frame validated against one schema:

Column	Meaning
`date`	trading date (parsed to `datetime64`)
`symbol`	instrument identifier
`open, high, low, close`	prices; `close` is adjusted where the source provides it
`volume`	traded volume

`DataSource._validate` enforces the schema, sorts by `(symbol, date)`, and rejects non-positive prices before any downstream code sees the frame. `qp.data.store.load_prices` wraps a source with a content-addressed parquet cache: the config's data section is hashed, and a repeat run with an identical configuration reads from disk instead of re-fetching.

2.2 Four sources, one always working

Source	Notes
`synthetic`	default; zero dependencies, zero network, deterministic given a seed. Used by every test and the demo.
`csv`	one file per symbol under `data.csv_dir`; the robust path for real historical data (e.g. an ENTSO-E export).
`yfinance`	guarded optional import (`pip install "qp[data]"`); degrades to a clear `ImportError` rather than a broken run.
`stooq`	stdlib-only HTTP fetch; best-effort, since stooq's endpoint has started serving a bot-check page from some networks.

Table 2.1: The pipeline never depends on network access to run its tests or its demo.

2.3 The synthetic generator

Real market data cannot be committed to a repository and real free APIs are unreliable enough to break a demo. `qp.data.synthetic.SyntheticSource` solves both problems and, deliberately, does one more job: it injects a *known, controllable* amount of predictable structure so the rest of the pipeline has something real to find.

For k symbols over n trading days, log-returns follow a correlated geometric Brownian motion with a slow-moving, per-symbol drift component:

$$r_t = \left(\mu_t + \alpha_t - \tfrac{1}{2}\sigma_t^2\right)\Delta t + \sigma_t\sqrt{\Delta t}\, L\, z_t, \qquad z_t \sim \mathcal{N}(0, I_k) \tag{2.1}$$

$$\alpha_t = \phi\, \alpha_{t-1} + \varepsilon_t, \qquad \varepsilon_t \sim \mathcal{N}(0, \eta^2 I_k) \tag{2.2}$$

where $\Delta t = 1/252$, L is the Cholesky factor of a correlation matrix with off-diagonal ρ (a one-factor structure: every pair of symbols shares correlation ρ), and (μ_t, σ_t) follow a regime path that switches between hand-tuned *bull*, *bear*, and *chop* states (durations drawn from $\mathcal{N}(\text{mean length}, 0.25 \cdot \text{mean length})$, floored at 20 days). α_t is an AR(1) process per symbol with $\phi = 0.99$ (half-life ≈ 69 trading days) – the only genuinely forecastable structure in the data, and the reason a momentum or ML signal has anything real to capture. OHLC and volume are then simulated around each simulated close with independent intraday noise.

Parameter	Demo value
μ (baseline drift)	6%/year, regime-modulated
σ (baseline vol)	20%/year, regime-modulated
ρ (pairwise correlation)	0.3
ϕ (alpha persistence)	0.99
η, alpha scale	0.08 innovation std, ×0.22 overall

Table 2.2: `configs/demo.yaml -> data.synthetic`. Every value is a config knob; `qp demo` fixes them to produce a reproducible Sharpe 0.62 for cross-sectional momentum (Chapter 8).

The alpha scale (0.22) is chosen so the demo is legible, not because it resembles a real single-name equity anomaly. A genuine, persistent, easily-captured 69-day alpha of this magnitude does not exist in liquid public markets; if it did, it would already be arbitraged away. Treat every number in Chapter 8 as a statement about the code, not about markets.

CHAPTER 3

Feature Engineering

3.1 Causal, token-resolved indicators

Every function in `qp.features.technical` takes a wide close panel (index = date, columns = symbol) and returns a same-shaped panel where row t depends only on data up to and including t – there is no indicator in this module that can see the future.

Token	Formula	Reads
`ret_n`	$\ln P_t - \ln P_{t-n}$	n-day log return
`vol_n`	$\mathrm{std}_n(\Delta \ln P)\sqrt{252}$	annualised realised volatility
`mom_n`	$P_{t-\mathrm{skip}}/P_{t-n-\mathrm{skip}} - 1$	trailing return, skipping the most recent `skip` days (avoids the well-known 1-week reversal contaminating a momentum signal)
`zscore_n`	$(P_t - \mathrm{MA}_n)/\mathrm{SD}_n$	distance from the rolling mean, in standard deviations
`rsi_n`	$100 - 100/(1+RS)$, $RS = \overline{\Delta^+}_n / \overline{\Delta^-}_n$	Wilder's relative strength index
`macd`	$\mathrm{EMA}_{12} - \mathrm{EMA}_{26}$, minus its own 9-day EMA	MACD histogram
`bbp_n`	$(P_t - L_n)/(U_n - L_n)$, $U/L = \mathrm{MA}_n \pm 2\,\mathrm{SD}_n$	%B: position within the Bollinger band
`skew_n`	rolling skewness of log-returns	tail asymmetry

Table 3.1: Token grammar: `"<base>_<window>"` resolves via a small registry in `compute()`; a config just lists tokens like `mom_60, rsi_14`.

3.2 Building an ML dataset without leaking the future

`build_ml_dataset(close, tokens, horizon)` produces a tidy (X, y) pair indexed by `(date, symbol)`:

```
fwd = close.shift(-horizon) / close - 1.0 # row t: return realised over (t, t+horizon]
y = fwd.stack()
X = pd.concat([compute(tok, close).stack() for tok in tokens], axis=1)
df = X.join(y, how="inner").dropna()
```

Two guarantees follow directly from this construction and are exercised by `tests/test_data_features.py::te`

1. every feature at row (t, s) is computable from data available at the close of day t;
2. the label at row (t, s) is a *forward* return, so the final `horizon` dates in the sample have no label and are dropped – a model trained on this frame cannot see its own target.

Design rationale

The no-leakage property is enforced structurally (by the shift and the inner join), not by convention or a code-review checklist. This is the single most common way a backtest lies to its author, so it is worth making impossible rather than merely discouraged.

CHAPTER 4

Signal Models

4.1 The contract

Every signal implements `generate(close) -> DataFrame` of scores, same shape as `close`. A score is a "desire to hold" in roughly $[-1, 1]$; the portfolio layer (Chapter 6) turns scores into leverage- and risk-constrained weights. A signal marks itself `market_neutral = True` when it already produces cross-sectionally demeaned scores.

4.2 Rule-based signals

Signal	Score
`Momentum`	$\tanh\left(\text{mom}_n / \overline{\lvert\text{mom}_n\rvert}_{252}\right)$, optionally divided by realised vol so noisier names are sized down
`MeanReversion`	$-z_n/(2 \cdot \text{entry_z})$, clipped to $[-1, 1]$: fade a stretched short-horizon z-score
`CrossSectionalMomentum`	cross-sectional z-score of mom_n, renormalised each day so $\sum_s \lvert w_s\rvert \approx 1$: long the relative winners, short the relative losers, market-neutral

`CrossSectionalMomentum` is the pipeline's default and the one used for the headline demo result – it is the closest rule-based analogue of the persistent per-symbol α_t injected into the synthetic data (Chapter 2), so it is the fairest baseline for the ML comparison in Chapter 8.

4.3 A walk-forward ML signal

`MLSignal` wraps a `scikit-learn` pipeline (`StandardScaler + HistGradientBoostingRegressor` by default) and trains it *only* on the walk-forward splits described in Chapter 5: for each `(train_start, train_end, test_start, test_end)` window it fits on the training rows of the leak-free dataset from Chapter 3, predicts on the test rows, and stitches the out-of-sample predictions back into a wide panel. The cross-sectional z-score of that prediction becomes the score, exactly as in `CrossSectionalMomentum` – so the two signals are compared on equal footing in Chapter 8: same sizing, same costs, same risk limits, different alpha source.

4.4 An optional sequence model

`SequenceSignal` is a single-layer LSTM over a rolling window of the same feature tokens, trained walk-forward with the same split logic:

It requires `torch` and is import-guarded: without the `ml` extra installed, selecting `signal.type: sequence` raises a clear `ImportError` naming the alternatives, rather than crashing deep inside a training loop.

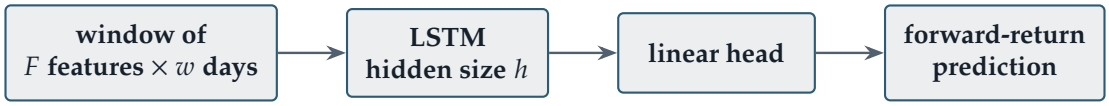


Figure 4.1: One (symbol, date) training example: the last w days of the feature panel for that symbol, target = forward return over `horizon` days. The final hidden state feeds a linear head; only the last time step's output is used (`out[:, -1, :]`).

This is included to *learn the architecture and benchmark it honestly*, not because it is expected to win. Chapter 10 explains why an LSTM over technical features is dominated by transaction costs and regime shift on real data far more often than it earns its keep. Never trust a sequence model's in-sample equity curve.

CHAPTER 5

Validation Methodology

Standard K-fold cross-validation assumes exchangeable, independent samples. Financial time series are neither: adjacent rows overlap (a 5-day forward return at t and at $t+1$ share four days of data), so a naive random split leaks the future into the training set through that overlap. `qp.validation` implements two splitters that respect time ordering and remove the overlap.

5.1 Walk-forward

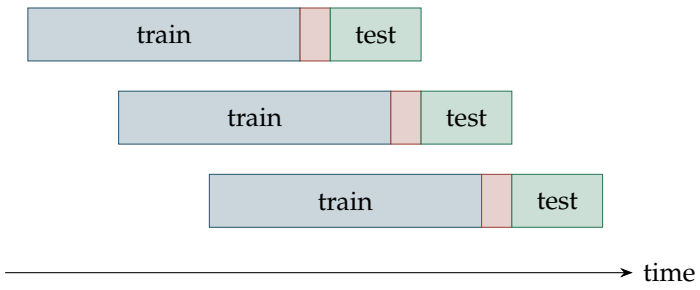


Figure 5.1: `walk_forward(dates, train_days, test_days, step_days, embargo_days)`. The window slides forward by `step_days` each iteration; the red embargo gap between train and test is at least as long as the label horizon, so no training row's label overlaps a test-period date.

`MLSignal` and `SequenceSignal` both consume this splitter directly. For each window they train fresh (no state carried across windows – a deliberate choice: it exercises the model's ability to relearn from only recent regime data, which is closer to how a live system would operate) and predict on the test window only; the concatenation of all test predictions is the out-of-sample score series used everywhere downstream.

5.2 Purged K-fold

`purged_kfold(dates, n_splits, embargo_days)` implements the Lopez-de-Prado-style alternative: split the date range into K contiguous test folds, and for each fold drop every training row within `embargo_days` of that fold's boundary on either side. This is available for parameter studies where multiple folds are needed cheaply; the pipeline's default (`validation.scheme: walk_forward`) is the more realistic choice for anything that will inform a live decision, because it never trains on data from after the period it is being evaluated on – purged K-fold's later folds can still train on data that is chronologically after an earlier test fold, which walk-forward never does.

Design rationale

The embargo is not cosmetic. With a 5-day forward-return target and no embargo, a model can partially "see" test-period information through overlapping labels at the train/test boundary – a subtle leak that inflates backtested Sharpe without ever showing up as an obvious bug. Setting `embargo_days` $\geq$ the label horizon closes it structurally.

CHAPTER 6

The Backtest Engine

6.1 Timeline convention

The single most common source of an inflated backtest is look-ahead: using information at bar t that would not actually have been known at bar t. The engine enforces one convention everywhere:

1. scores and target weights for bar t are computed from data available *at the close* of bar t;
2. those weights are *held over* $t \to t+1$ (a one-bar execution lag: `held = target_w.shift(1)`);
3. realised return over that bar is `held * asset_return`, i.e. yesterday's decision earns today's return, never today's decision earning today's return.

`tests/test_backtest.py::test_no_lookahead_one_bar_lag` encodes this as a unit test: a score that only exists on the final bar of the sample can have no effect on that bar's return, because there is no bar $t+1$ to hold it into.

6.2 Position sizing

Three sizing schemes share one pipeline: build raw scores, cap per name, project to long-only if configured, then renormalise to the target gross leverage.

Fixed. Weights proportional to score, renormalised to `gross_leverage`. No volatility adjustment.

Risk parity. $w_s \propto \text{sign}(\text{score}_s)/\sigma_s$ before the same renormalisation – equal risk contribution per name under the (strong) assumption of no cross-correlation.

Volatility targeting. The book is scaled each day so its *ex-ante* volatility tracks a target:

$$\sigma_{\text{port},t} = \sqrt{w_t^\top \Sigma_t \, w_t}\,\sqrt{252}, \qquad w_t \leftarrow w_t \cdot \min\left(\frac{\sigma_{\text{target}}}{\sigma_{\text{port},t}},\ \kappa_{\max}\right) \tag{6.1}$$

where Σ_t is a rolling sample covariance of daily returns (`vol_lookback` days) and $\kappa_{\max}$ (`max_leverage_scale`) caps the multiplier so a quiet-market estimate of Σ_t cannot imply arbitrary leverage. This is computed from the *portfolio* covariance, not a sum of individual-name volatilities – it therefore correctly rewards diversification and long/short netting: a market-neutral book with two highly correlated offsetting legs is recognised as low-risk, which a naive $\sum_s |w_s|\sigma_s$ proxy would not capture.

Config knob	Effect
`target_vol`	annualised target for σ_{port}
`max_weight`	per-name cap, applied before and after vol scaling
`gross_leverage`	the leverage the raw (pre-vol-scale) book normalises to
`vol_lookback`	window for Σ_t
`max_leverage_scale`	ceiling on the vol-target multiplier κ
`rebalance`	`daily` / `weekly` / `monthly`; weights hold between rebalance dates

6.3 Cost model

Turnover is the L^1 change in *target* weight between consecutive bars, $\tau_t = \sum_s |w_{s,t} - w_{s,t-1}|$. Costs charged against gross return:

$$\text{transaction cost}_t = \tau_t \times \frac{\text{commission}_{\text{bps}} + \text{spread}_{\text{bps}} + \text{slippage}_{\text{bps}}}{10^4} \tag{6.2}$$

$$\text{borrow cost}_t = \Big(\sum_{s:\, w_{s,t}<0} |w_{s,t}| \Big) \times \frac{\text{borrow}_{\text{bps/yr}}}{10^4 \times 252} \tag{6.3}$$

Net return is `gross - transaction_cost - borrow_cost`. The tearsheet (Chapter 7) plots the cumulative decomposition of all three terms so a strategy's apparent edge can be checked against how much of it costs eat.

6.4 Risk limits and the drawdown kill-switch

Pre-trade, `risk.limits.apply_limits` clips each name to `max_position` and rescales the whole book if gross exposure exceeds `max_gross`. Separately, the engine runs a drawdown kill-switch on the realised net-return series:

```
track running equity and its peak; the moment drawdown breaches max_drawdown_stop,
flatten the book (zero return, zero weight) for a fixed cooldown (21 trading days
by default); reset the peak on resume.
```

This models a real risk desk's behaviour more faithfully than an instantaneous recovery rule would: after a real stop-out, a desk does not resume risk the instant equity happens to tick back up – it sits out a cooldown period and restarts clean.

6.5 Metrics

`qp.backtest.metrics.summary` reports, from the daily net-return series r_t (252 trading days/year):

$$\text{CAGR} = \Big(\prod_t (1 + r_t) \Big)^{252/N} - 1 \qquad \text{Sharpe} = \sqrt{252}\, \frac{\overline{r_t - r_f/252}}{\text{sd}(r_t - r_f/252)}$$

$$\text{Sortino} = \sqrt{252}\, \frac{\overline{r_t - r_f/252}}{\text{sd}\big((r_t - r_f/252)^-\big)} \qquad \text{Calmar} = \frac{\text{CAGR}}{|\text{max drawdown}|}$$

$$\text{VaR}_{95} = Q_{0.05}(r_t) \qquad \text{CVaR}_{95} = \mathbb{E}[r_t \mid r_t \le \text{VaR}_{95}]$$

plus hit rate, skew, kurtosis, best/worst day, and – when a weights panel is supplied – average gross/net exposure, average daily turnover, and time in market. This is the same dictionary written to `metrics.json` for every run and read back by the MCP `list_runs` tool (Chapter 9).

CHAPTER 7

Execution and the Live Loop

7.1 The broker contract

`qp.execution.base.Broker` is four methods: `equity()`, `positions()`, `mark(prices)`, `rebalance_to(target_weights, prices)`. The live loop only ever thinks in target weights; a broker implementation is responsible for turning a weight vector into orders against its own current positions.

7.2 The paper broker

`PaperBroker` is the default and, at this stage of the project, the only broker that actually works. It holds cash and per-symbol share quantities in memory, fills at the supplied price plus a symmetric slippage in basis points, charges a commission, and – critically – persists its full state (`cash`, `positions`, `last_prices`, the last 500 fills) to `execution_state/<name>.json` after every rebalance:

```
target_qty = (weight * equity) / price
delta = target_qty - current_qty
fill_price = price * (1 + sign(delta) * slippage_bps / 1e4)
commission = abs(delta * fill_price) * commission_bps / 1e4
cash -= delta * fill_price + commission
```

Because state is persisted, a fresh `PaperBroker(name=...)` constructed later – in a new process, after a reboot, from a cron job – resumes exactly where the last one left off. `tests/test_pipeline_exec.py::test_paper_broker_rebalance_and_resume` exercises this directly: two independently constructed broker instances sharing a state directory see the same equity.

7.3 The live loop

The loop reuses the exact `load_prices` / `make_signal` / `build_weights` / `apply_limits` functions the backtester uses (Chapter 1) – there is one implementation of "turn a signal into a target book," not two that could silently diverge.

7.4 The real-broker stub

`execution/alpaca.py` exists and raises `NotImplementedError` on construction, with the wiring instructions in its docstring. This is deliberate, not an oversight: the point at which paper trading has produced months of consistent, believable results is the point at which wiring a live broker is a conscious decision, not an accident of a config flag left on the wrong value. Every bundled config defaults to `execution.broker: paper`.

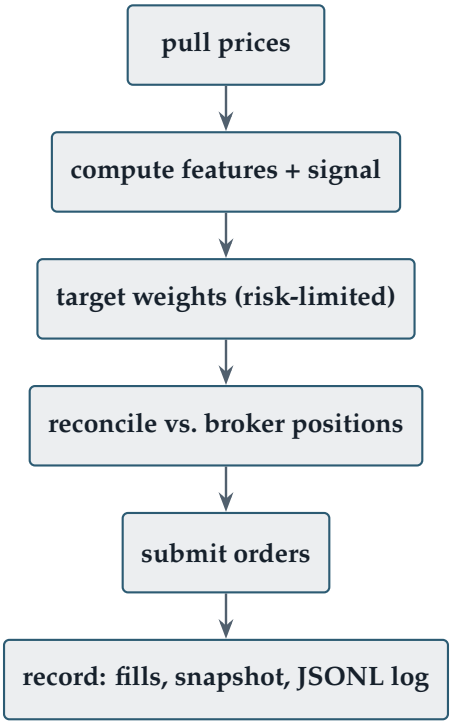


Figure 7.1: `qp.live.loop.run_once(cfg)`. Safe to call from cron: each call is a self-contained iteration that ends by writing one JSON line to `execution_state/<name>.loop.jsonl` and the broker's own state file. `run_forever` is a thin `time.sleep` wrapper around the same function for a long-running process.

CHAPTER 8

Results on the Bundled Demo

Reproduce every number in this chapter with:

```
qp demo
qp walkforward -c configs/demo.yaml
```

8.1 The demo run

`configs/demo.yaml`: 8 synthetic symbols, 2013–2024, `cross_sectional_momentum` (60-day lookback, 5-day skip), volatility-targeted at 10%/year, weekly rebalance, 1 bp commission + 2 bp spread + 50 bp/year borrow.

Metric	Value	Metric	Value
Trading days	3,131	Hit rate	50.7%
CAGR	4.57%	Skew / kurtosis	0.08 / 2.87
Annualised vol	7.73%	Daily VaR_{95} / $CVaR_{95}$	−0.74% / −1.07%
Sharpe	0.62	Avg. gross / net exposure	1.34 / −0.03
Sortino	0.92	Avg. daily turnover	0.10
Max drawdown	−14.87%	Time in market	97.8%
Calmar	0.31		

Table 8.1: `runs/demo/metrics.json`, reproduced exactly by `qp demo` (fixed seed). Net exposure near zero confirms the book is running market-neutral, as `CrossSectionalMomentum` is designed to.

8.2 Rule versus ML, same data, same costs

`qp walkforward` runs `cross_sectional_momentum` and the walk-forward `MLSignal` (a `HistGradientBoostingRegress` on the same feature set, 5-day forward-return target) through the identical sizing, cost, and risk pipeline, on the identical dataset:

Variant	Sharpe	CAGR	Max drawdown	Avg. daily turnover
`cs_momentum`	**0.62**	4.6%	−14.9%	0.10
`ml_gbr`	−0.07	−0.7%	−21.1%	0.17

Table 8.2: Same universe, same walk-forward discipline, same cost model. The simple rule wins, and by a wide margin.

A second, independent dataset (`configs/ml_walkforward.yaml` – 10 years, 8 different synthetic symbols, a fresh seed) gives the ML signal Sharpe −0.57 on its own – consistent with the

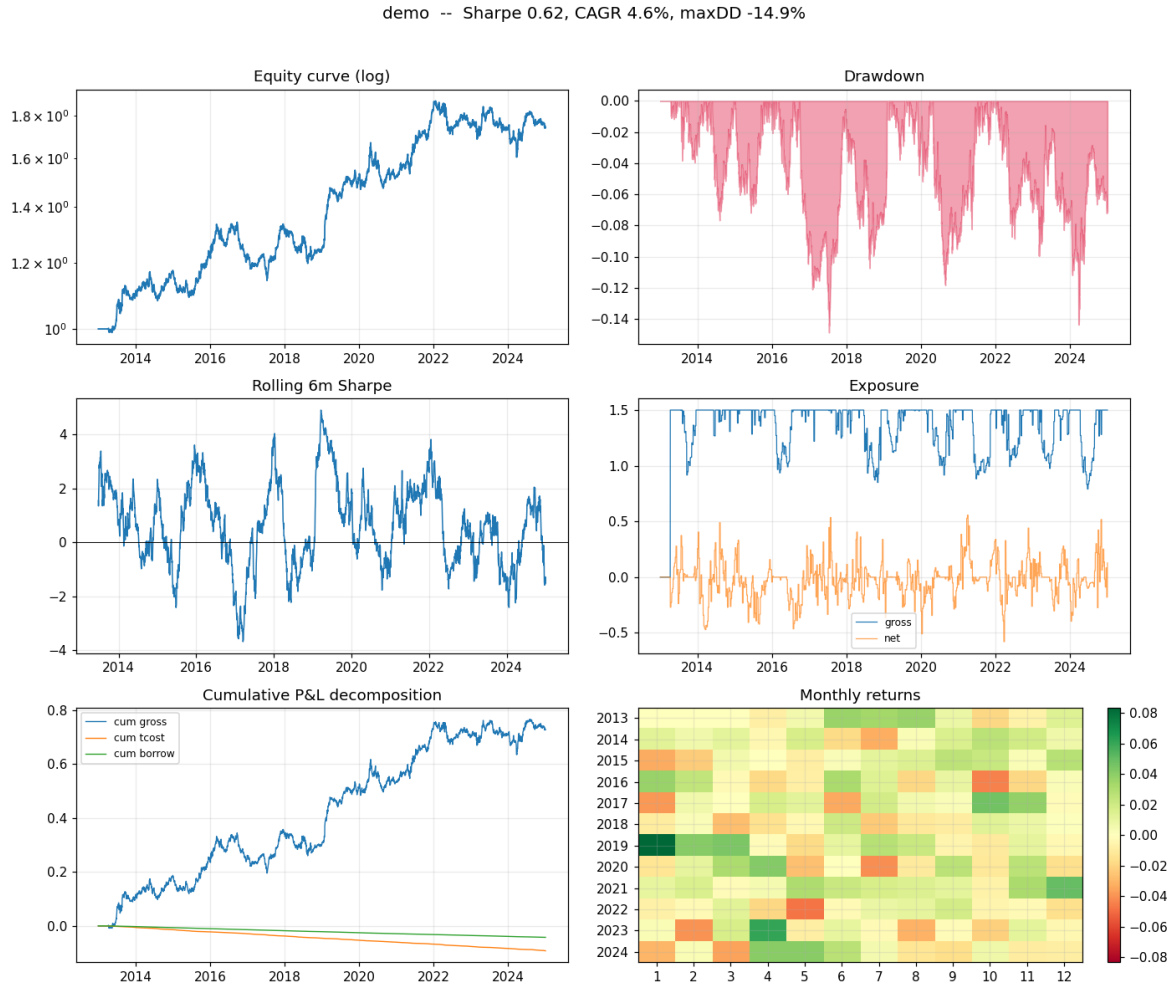


Figure 8.1: `runs/demo/tearsheet.png`: equity curve, drawdown, rolling 6-month Sharpe, gross/net exposure, cumulative P&L decomposition (gross vs. transaction cost vs. borrow cost), and the monthly-return heatmap.

comparison above, not a fluke of one run.

8.3 Why the rule beat the model here

The synthetic alpha (Chapter 2) is a slow AR(1) process with a ≈69-day half-life. A 60-day trailing-return momentum signal is close to a matched filter for exactly that process. The ML model is asked to recover the same signal from a 5-day-ahead target through a gradient-boosted tree over noisy technical features – a much harder estimation problem, with far more ways for the model to fit noise in the training window that does not repeat in the test window. This is not a defect in the implementation; it is the expected outcome whenever a flexible model is pointed at a low signal-to-noise, slowly-varying target without a structural reason to expect it to generalise better than a simple, correctly-shaped rule. See Chapter 10 for the general version of this argument and its implications for real markets.

CHAPTER 9

The Agent Layer: MCP, and What's Next

9.1 Why an MCP server on top of a CLI

The CLI (`qp demo`, `qp backtest`, `qp walkforward`, `qp paper`) is sufficient for a human at a terminal. An agent needs the same capabilities as typed, discoverable tool calls rather than a stream of shell output to parse – and once the pipeline speaks MCP, it is reachable from Claude Code, Claude Desktop, or any other MCP client (including, per the brief this project started from, an orchestration layer like Google Stitch) without writing a second integration.

9.2 What is exposed today

Tool	Effect	Side effects
`describe_pipeline()`	returns this project's README	none
`list_configs()`	lists `configs/*.yaml`	none
`list_runs()`	lists `runs/*` with Sharpe/CAGR/maxDD	none
`run_backtest(config_path)`	full data→features→signal→backtest→tearsheet	writes `runs/<name>/`
`get_run_report(name)`	plain-text metrics table	none
`get_tearsheet_path(name)`	absolute path to the PNG	none
`paper_iteration(config_path)`	one live-loop step against the paper broker	writes `execution_state/`

Table 9.1: `mcp/server.py`, built on `FastMCP`. Registered for Claude Code via the project's `.mcp.json`. Nothing here can reach a real broker (Chapter 7).

The server degrades gracefully: without the `mcp` package installed, it prints install instructions to stderr and exits 0 rather than raising inside an agent's tool-discovery call.

9.3 Roadmap

1. **Now.** Read-mostly tools above; an agent proposes configs and reads results.
2. **Next – `propose_signal(description)`.** The calling agent drafts a new `Signal` subclass from a natural-language description, drops it into `src/qp/signals/`, registers it, and backtests it against the existing baselines in one call.
3. **Next – `sweep(param_grid)`.** A walk-forward parameter study across a grid (e.g. `lookback` $\in \{20, 60, 120\}$), returning a ranked table – the mechanical half of strategy research, delegated.
4. **Next – `research_report(run_a, run_b)`.** Diff two runs' metrics, weights, and cost decomposition into a short markdown memo.

5. **Later – real data & a real signal.** Swap the `csv` source for an ENTSO-E day-ahead export and add a power-market `Signal` (features: load, wind/solar, and fuel-price forecasts; target: the day-ahead↔intraday spread, or a storage-dispatch P&L). This reuses the PyPSA / OR-Tools background directly and is the artefact described in Chapter 10 as the actual point of this project.

CHAPTER 10

Honest Limitations

This chapter exists because a backtester that only shows its wins is the most common way a systematic-trading project deceives its own author. Read it before reading Chapter 8 as encouragement rather than as a demonstration of technique.

10.1 Everything in Chapter 8 is synthetic

The demo's Sharpe of 0.62 is measured against data generated by this project's own `SyntheticSource`, tuned (Chapter 2) so the momentum signal has something real to find. It says the plumbing is correct – data flows through features, signals, validation, costs, and risk without leaking the future or silently dropping days. It says *nothing* about whether momentum, or any other signal in this repository, would make money on real markets. No backtest on synthetic data can say that, by construction.

10.2 Retail systematic trading is a near-zero-sum game

Even against real data, an individual running this pipeline is competing against firms with colocated servers, direct exchange feeds, proprietary alternative data, and teams of specialists across the four roles described in the companion research note (`quant-trading-reality.md`). None of that can be replicated by adding more code to this repository. The honest reading of "can I build my own code and profit from it" is: not reliably, and the expected value of trying, net of the hundreds of hours it takes, is close to zero to negative for most people, most of the time.

10.3 Why the LSTM (and the GBR) should not be trusted here

- **Low signal-to-noise.** Even the synthetic alpha, deliberately exaggerated, produces a modest Sharpe once realistic costs are charged. Real single-name equity alphas are far weaker and far more crowded.
- **Non-stationarity.** A model trained on one regime (Chapter 2's bull/bear/chop states) has no guarantee its learned relationship survives into the next regime – and real markets change regime on no fixed schedule.
- **Multiple-comparisons risk.** Trying enough feature sets, architectures, and hyperparameters against the same historical window will eventually produce an attractive-looking backtest by chance alone. Walk-forward discipline (Chapter 5) reduces this but does not eliminate it if the model itself, or its hyperparameters, were chosen by looking at walk-forward results across many attempts.
- **Costs are not an afterthought.** Chapter 8 shows the ML variant had *higher* turnover (0.17 vs. 0.10) as well as worse gross performance – a common pattern: models that chase noisy short-horizon signals trade more and pay more in the process.

10.4 What this project is actually worth

1. **A portfolio artefact.** A clean backtester, an honest walk-forward ML comparison, and a paper-trading loop are exactly the kind of work sample an energy or commodity trading desk wants to see from a candidate, independent of whether the included strategies are profitable.
2. **A base for the real next step.** A power-market signal (Chapter 9) built on this same infrastructure – honest costs, honest validation, no look-ahead – is a genuinely useful research tool, because power markets have structural, physically-grounded drivers (weather, load, plant outages, storage economics) that are a fundamentally different estimation problem from "predict next week's equity return from its own price history."
3. **Market literacy that compounds.** Building the cost model, the vol-targeting formula, and the walk-forward splitter is a faster way to internalise how markets and risk actually work than reading about it.

If a future version of this pipeline, on real data, produces a consistently positive out-of-sample Sharpe after realistic costs: be more suspicious, not less. Check for a data bug, a leak, or a benchmark that was quietly easy before concluding the strategy is real.

APPENDIX A

Configuration Reference and File Map

A.1 `RunConfig` schema

Section	Key fields	Notes
`data`	`source, symbols, start, end, synthetic.*, cache`	source ∈ {synthetic, csv, yfinance, stooq}
`features`	a list of tokens	e.g. `[ret_1, vol_20, mom_60, rsi_14]`
`signal`	`type, params`	type ∈ {momentum, mean_reversion, cross_sectional_momentum, ml, sequence}
`portfolio`	`sizing, target_vol, max_weight, gross_leverage, long_only, rebalance, vol_lookback, max_leverage_scale`	sizing ∈ {fixed, vol_target, risk_parity}
`costs`	`commission_bps, half_spread_bps, borrow_bps_annual, slippage_bps`	all in basis points
`risk`	`max_drawdown_stop, max_gross, max_position, vol_lookback`	drawdown stop = 0 disables the kill-switch
`validation`	`scheme, train_days, test_days, step_days, embargo_days, n_splits`	consumed by `ml` / `sequence` signals
`execution`	`broker, starting_equity, state_dir`	broker ∈ {paper, alpaca (stub)}
`seed`	one integer	propagates to `data.synthetic.seed` and every model's RNG

A.2 File map

Path	Contents
src/qp/config.py	pydantic models, one per stage; RunConfig.from_yaml
src/qp/data/	base.py (contract), synthetic.py, csv_source.py, remote.py (yfinance/stooq), store.py (parquet cache)
src/qp/features/	technical.py (indicators + token registry), pipeline.py (panels, ML dataset)
src/qp/signals/	base.py, rules.py, ml.py, sequence.py, factory in __init__.py
src/qp/validation/splits.py	walk_forward, purged_kfold
src/qp/backtest/	engine.py, portfolio.py, costs.py, metrics.py
src/qp/risk/limits.py	pre-trade weight limits
src/qp/execution/	base.py, paper.py, alpaca.py (stub)
src/qp/report/tearsheet.py	PNG + text report + metrics.json + parquet artefacts
src/qp/live/loop.py	run_once, run_forever
src/qp/pipeline.py	the orchestrator used by the CLI
src/qp/cli.py	qp demo / backtest / data / walkforward / paper / report
mcp/server.py, mcp/README.md	the agent layer and its design notes
configs/demo.yaml, configs/ml_walkforward.yaml	runnable configs referenced throughout this document
tests/	13 tests: data/feature integrity, no-lookahead, cost drag, drawdown stop, paper-broker persistence, end-to-end pipeline

A.3 Reproducing this document's numbers

```
cd quant-pipeline
pip install -e .
make test                    # 13 passed
qp demo                     # Chapter 8, Table 8.1 + Figure 8.1
qp walkforward -c configs/demo.yaml # Chapter 8, Table 8.2
```